

Mini Packet Router / Telemetry Mux

Side Project for Bit Manipulation & Concurrency Practice

Design Document — Living Draft

Pablo Buceta Santos

September 27, 2026

Abstract

This document describes a small, self-contained C side project whose purpose is deliberate practice of two interview-relevant skills: bit-level packing/unpacking/validation and concurrent programming with shared state in C. It is intentionally independent of the MiniFlight simulation. The document is living: it records the current design intent, the rationale behind decisions, and a short catalogue of related concepts that are important for flight-software interviews but are deliberately kept out of the implementation scope of this project.

Contents

1	Document status and scope	2
1.1	What this project is	2
1.2	What this project is not	2
1.3	Relationship to MiniFlight	2
1.4	Maturity	2
2	Goals and non-goals	2
2.1	Primary goals	2
2.2	Secondary goals	3
2.3	Explicit non-goals	3
3	High-level architecture	3
3.1	Why a ring buffer of fixed slots	3
3.2	Thread counts (starting point)	3
4	Packet layout and bit manipulation	4
4.1	Packet size	4
4.2	Logical fields (illustrative; exact bit positions still open)	4
4.3	Required bit operations	4
4.4	Validation rules (consumer side)	4
5	Concurrency model	5
5.1	Baseline (always present)	5
5.2	Optional modules (add one at a time)	5
5.3	Design rationale for the baseline choices	5
6	Shared status and observability	5
7	Implementation order	6

8 Interview-relevant concepts deliberately kept out of scope	6
8.1 Real-time task scheduling	6
8.2 Priority inversion and priority inheritance / ceiling	6
8.3 Hard real-time operating systems	6
8.4 Other concurrency topics of secondary priority for this project	7
9 Design decision log (initial)	7
10 How to maintain this document	7
11 Revision note	8

1 Document status and scope

1.1 What this project is

A focused practice vehicle for:

- Bit manipulation on fixed-size packets (encode, decode, validate).
- Classic concurrency patterns in C (producer–consumer, ring buffer, mutexes, condition variables, limited atomics, ownership transfer).

It is a learning and interview-preparation artefact, not flight software and not part of MiniFlight.

1.2 What this project is not

- Not a real-time operating system or a cyclic executive.
- Not hard real-time; it runs on ordinary Linux with pthreads.
- Not a network stack, device driver, or hardware interface.
- Not an attempt to replace or extend the MiniFlight scheduler.
- Not a complete concurrency curriculum (see Section 8 for deliberately omitted topics).

1.3 Relationship to MiniFlight

MiniFlight remains single-threaded by design for the foreseeable future. This side project exists precisely so that concurrency and bit-level practice can be pursued without forcing a premature redesign of the main simulation.

1.4 Maturity

As of this draft the project exists only as a design. Implementation will proceed in the order given in Section 7. When code appears, this document should be updated to match reality (present-tense architecture only for what is actually built).

2 Goals and non-goals

2.1 Primary goals

1. Become fluent with common bit-manipulation patterns used on status words, flags, and tightly packed telemetry/command fields.
2. Understand and be able to explain the core concurrency problems that appear when multiple threads share a queue and a statistics structure: data races, critical sections, ownership, full/empty conditions, and safe shutdown.
3. Produce a small, readable program that can be run under load so that races and validation failures are observable.
4. Keep the code size modest (target: a few hundred lines) so that the design remains fully understandable.

2.2 Secondary goals

- Practice writing clear rationale for design choices (recorded in this document).
- Maintain a short reference list of interview-relevant concepts that are *not* implemented here, so that study time can be allocated deliberately.

2.3 Explicit non-goals

- Implementing a real-time scheduler, rate-monotonic priority assignment, Liu & Layland test, or response-time analysis inside this program.
- Hard real-time behaviour or deadline monitoring.
- Lock-free queues as the default path (an optional atomic counter is acceptable; a full lock-free ring is not required).
- Dynamic memory allocation on the steady-state path.
- Integration with MiniFlight components or message IDs.

3 High-level architecture

The program is a concurrent packet pipeline:

Producers	Shared ring buffer	Consumers
Build fixed-size packets with packed bit-fields; push into ring.	Fixed capacity circular buffer of packet slots; head/tail protected.	Pop packets, decode bits, validate, update shared status, release slot.

An optional low-frequency monitor thread periodically samples the shared status word and prints a human-readable summary. This makes correctness (or the lack of it) visible under load.

3.1 Why a ring buffer of fixed slots

- Matches the “fixed pools, no heap on the hot path” bias common in flight software.
- Forces explicit ownership transfer of each slot.
- Makes full/empty conditions obvious and therefore good practice for condition variables or equivalent.
- Keeps memory layout simple for bit-level work on the packet contents.

3.2 Thread counts (starting point)

- 2 producers, 1 consumer (baseline).
- Later: 3+ producers, 2 consumers to increase contention.
- Optional monitor thread.

Exact numbers should be configurable at compile time or via command line so that races can be provoked and then eliminated.

4 Packet layout and bit manipulation

4.1 Packet size

Fixed at 16 or 32 bytes (decision to be frozen before implementation). Treated as an array of `uint8_t`; multi-byte fields may be accessed via carefully aligned `uint32_t`/`uint64_t` views or by explicit byte shifts. Compiler bit-fields are avoided for the core practice so that shifts and masks remain visible.

4.2 Logical fields (illustrative; exact bit positions still open)

- Magic / version (must match a constant).
- Source / producer ID (small integer, range-checked).
- Sequence number (16- or 32-bit, wraps).
- Flags (individual bits or small bit-fields: priority, error, end-of-group, ...).
- One or two payload values of varying bit width.
- Reserved bits (must be zero on a valid packet).
- Parity or simple checksum over selected fields (1–8 bits).

4.3 Required bit operations

Producers and consumers must exercise:

- Set, clear, test single bits and contiguous bit ranges.
- Extract and insert multi-bit fields at arbitrary offsets.
- Pack several small values into one word.
- Compute and check a trivial parity or checksum.

4.4 Validation rules (consumer side)

A packet is rejected if any of the following fail:

- Magic / version mismatch.
- Reserved bits not zero.
- Source ID outside allowed range.
- Parity / checksum mismatch.
- Cross-field consistency rule (example: a given flag set implies a payload field non-zero).

Optional later extension: sequence-number monotonicity or gap detection against a shared last-seen value.

On validation failure the consumer still releases the slot (no leak) and updates error counters / sticky flags in the shared status word.

5 Concurrency model

5.1 Baseline (always present)

- One mutex protecting ring-buffer indices (or count).
- Condition variables (or equivalent) for “not full” and “not empty”.
- Clear ownership rule: a slot is owned by at most one thread at a time; ownership moves only under the mutex.
- Shared statistics / status word updated by consumers (and possibly read by the monitor) under appropriate protection.

5.2 Optional modules (add one at a time)

These are deliberately separable so that the baseline stays understandable.

1. **Second shared structure + second mutex.** Forces explicit lock-ordering discipline and makes deadlock a concrete risk that must be designed out.
2. **Reader–writer access to the status word.** Consumers write; monitor (and possibly producers) read. Realised with `pthread_rwlock` or a simple sequence-counter / version technique.
3. **Atomic fast path for a hot counter or flag.** C11 `_Atomic` (or `stdatomic.h`) so that the contrast between “mutex everywhere” and “atomic for the simple counter” can be discussed, including a high-level view of memory ordering.
4. **Clean shutdown / drain.** Global running flag, producers stop submitting, consumers drain the queue, all threads join. Surfaces lifetime and “who owns the last reference” questions.

5.3 Design rationale for the baseline choices

- Mutex + condition variables are the most common primitives interviewers expect a candidate to be able to use and explain.
- Starting with a single lock keeps the first correct version simple; additional locks are introduced only after the single-lock version is solid.
- Fixed-capacity ring avoids dynamic allocation and makes the full/empty conditions obvious.
- Copying the packet into the slot (or filling the slot in place under ownership) keeps lifetime reasoning straightforward.

6 Shared status and observability

A small shared structure (or a packed word plus a few counters) holds:

- Total packets processed (good + bad).
- Validation failure counts (optionally broken out by reason).
- Sticky error flags (bit-packed).
- Optionally last sequence number or a generation counter.

The monitor thread (or a consumer every N packets) prints a readable summary. At process exit a simple consistency check is performed (produced $\approx$ consumed + in-queue, counters non-negative, etc.).

7 Implementation order

1. Packet layout frozen; pure encode / decode / validate functions written and unit-tested single-threaded.
2. Ring buffer + single producer / single consumer with mutex and condition variables.
3. Multiple producers and consumers.
4. Shared status word and validation counters wired in.
5. One optional concurrency module at a time.
6. Stress runs and deliberate injection of malformed packets.

Each step should leave the program in a runnable, observable state.

8 Interview-relevant concepts deliberately kept out of scope

The following topics are important for flight-software interviews and for real avionics work, yet they are intentionally *not* implemented in this project. They should be studied in parallel (theory, paper exercises, or a separate tiny tool) so that conceptual fluency is not confused with the presence of code in this repository.

8.1 Real-time task scheduling

- Periodic tasks with period T , deadline D (usually $D \leq T$), and worst-case execution time C .
- Rate-monotonic priority assignment (higher rate $\rightarrow$ higher priority).
- Liu & Layland utilization bound as a sufficient (but not necessary) schedulability test.
- Response-time analysis (RTA) as the more precise fixed-point test.
- Practical consequences of a task set that fails the tests (missed deadlines, need to reduce rates or WCET, priority inversion, etc.).

Why out of scope here: implementing a scheduler, measuring WCET, enforcing deadlines, and handling preemption under a real RTOS is a separate project of non-trivial size. Linux pthreads with `SCHED_FIFO` give only a soft approximation. Conceptual mastery plus a few worked examples is higher leverage for interviews than coding the formulas inside this packet pipeline.

8.2 Priority inversion and priority inheritance / ceiling

Classic problem when a low-priority task holds a mutex needed by a high-priority task. Solutions (priority inheritance, priority ceiling) are standard RTOS features. This project may later illustrate the *problem* with two locks, but will not implement a priority-inheritance protocol.

8.3 Hard real-time operating systems

FreeRTOS, Zephyr, RTEMS, VxWorks, etc. Practice environments: real boards, QEMU, or vendor simulators. Stock Linux is not a hard RTOS; `PREEMPT_RT` improves latency but does not change the fundamental model.

8.4 Other concurrency topics of secondary priority for this project

- Full lock-free ring buffers and ABA problems.
- Memory-ordering deep dives beyond the level needed to use `_Atomic` correctly for simple counters.
- False sharing / cache-line padding (worth knowing about; not required to implement here).
- Signal safety and async cancellation.

9 Design decision log (initial)

Decision	Rationale
Separate project from Mini-Flight	Avoids contaminating the single-threaded design; keeps each artefact focused.
Fixed-size packets + ring of slots	Matches flight-software preference for static allocation; forces clear ownership.
Shifts/masks over compiler bit-fields	Makes the bit operations explicit and portable for practice.
Mutex + condvar as baseline	Highest-frequency interview primitives; easiest correct starting point.
Optional modules behind switches	Keeps baseline readable; allows incremental exposure to extra patterns.
Validation as first-class consumer work	Ties bit manipulation to a realistic robustness concern.
RT scheduler / RTA out of scope	Formulas are simple; surrounding scaffolding (WCET, release, observation, true RTOS) is not. Conceptual study is higher ROI for interviews.

10 How to maintain this document

Update this file when any of the following change:

Change	Section(s)
Packet field layout or validation rules frozen or altered	4
Ring capacity, ownership rules, or baseline synchronisation	3 , 5
Optional module added or behaviour clarified	5
Shared status contents or observability	6
Implementation status advances	1 , 7
New out-of-scope topic judged important for interviews	8

Prefer short factual edits. Planned but unimplemented features stay in non-goals or explicitly labelled “planned” paragraphs; they must not appear in present-tense architecture descriptions.

11 Revision note

Initial draft created before any implementation code. Records the agreed scope (bit manipulation + producer/consumer concurrency), the packet and ring-buffer shape, the optional concurrency modules, and the deliberate exclusion of a real-time scheduler / schedulability analysis from this particular project. Future revisions should track the code as it is written.